

llama.cpp 완전 정복 — 개념, 양자화 원리, 실습

GGML/GGUF 아키텍처와 블록 양자화의 수학적 원리부터, Hugging Face 공개 모델을 NVIDIA GeForce RTX 5060 Ti(16GB)에서 직접 변환·양자화·서빙하는 실습까지 다루는 심화 학습 문서.

대상	실습 환경	활용 모델
LLM 로컬 추론·양자화를 배우려는 엔지니어	RTX 5060 Ti 16GB · CUDA 12.8+ · Linux/WSL2	Qwen3 · Llama 3.1 · Gemma 3 · EXAONE 3.5

목차

01 llama.cpp 개요와 생태계

02 소프트웨어 아키텍처

03 GGUF 파일 포맷

04 양자화의 원리

05 실습 1 — 환경 구축과 빌드

06 실습 2 — 모델 다운로드와 첫 추론

07 실습 3 — HF 모델 변환과 양자화

08 실습 4 — llama-server와 OpenAI 호환 API

09 실습 5 — 성능 측정과 VRAM 튜닝

10 부록 — 추천 모델·명령어·트러블슈팅

01 llama.cpp 개요와 생태계

llama.cpp란 무엇인가

llama.cpp는 Georgi Gerganov가 2023년 3월 공개한 오픈소스 LLM 추론 엔진이다. 순수 C/C++로 작성되어 외부 런타임 의존성이 없고, 하나의 실행 파일로 노트북 CPU부터 데이터센터 GPU까지 동일한 모델 파일(GGUF)을 실행한다. 핵심 철학은 "양자화를 통한 접근성"이다 — FP16으로 16GB가 필요한 8B 모델을 4비트로 압축하면 5GB 남짓이 되어, RTX 5060 Ti 같은 소비자용 GPU 한 장으로 14B급 모델까지 무리 없이 실행할 수 있다.

Ollama, LM Studio, Jan, GPT4All 등 대부분의 로컬 LLM 도구가 내부적으로 llama.cpp를 엔진으로 사용한다. 따라서 llama.cpp를 이해하면 로컬 LLM 생태계 전체의 동작 원리를 이해하는 것과 같다.

핵심 특징

의존성 제로 C/C++ Python 런타임·프레임워크 없이 단일 바이너리로 동작. 임베딩·모바일까지 이식 가능.	폭넓은 양자화 (1.5~8비트) Q4_0 같은 레거시 타입부터 K-quants, IQ(i-quants), 중요도 행렬(imatrix) 기반 양자화까지 지원.
멀티 백엔드 CPU(AVX/NEON), CUDA, Metal, Vulkan, SYCL, HIP. 같은 GGUF 파일이 모든 백엔드에서 실행된다.	CPU-GPU 하이브리드 실행 레이어 단위 오프로딩(-ngl)으로 VRAM보다 큰 모델도 RAM과 나눠 실행.
OpenAI 호환 서버 내장 llama-server 하나로 /v1/chat/completions API와 웹 UI를 제공.	mmap 기반 즉시 로딩 GGUF는 메모리 맵 친화적 설계 — 파일을 복사하지 않고 페이지 단위로 매핑해 기동이 빠르다.

용어 정리: ggml · GGUF · llama.cpp

ggml은 텐서 연산 라이브러리(엔진의 심장), **GGUF**는 모델을 담는 파일 포맷(컨테이너), **llama.cpp**는 이 둘 위에 LLM 특화 로직(토큰나이저, KV 캐시, 샘플링, 채팅 템플릿)과 CLI·서버 도구를 엮은 프로젝트다. 셋의 관계는 2장의 구조도에서 명확해진다.

02 소프트웨어 아키텍처

llama.cpp는 4개 계층으로 구성된다. 위에서 아래로: 사용자가 실행하는 **도구 계층**, LLM 추론 로직을 담은 **libllama**, 범용 텐서 엔진 **ggml**, 하드웨어별 **백엔드**. 도구와 외부 바인딩(Python llama-cpp-python, Ollama 등)은 모두 llama.h라는 단일 C API를 통해 접근한다.

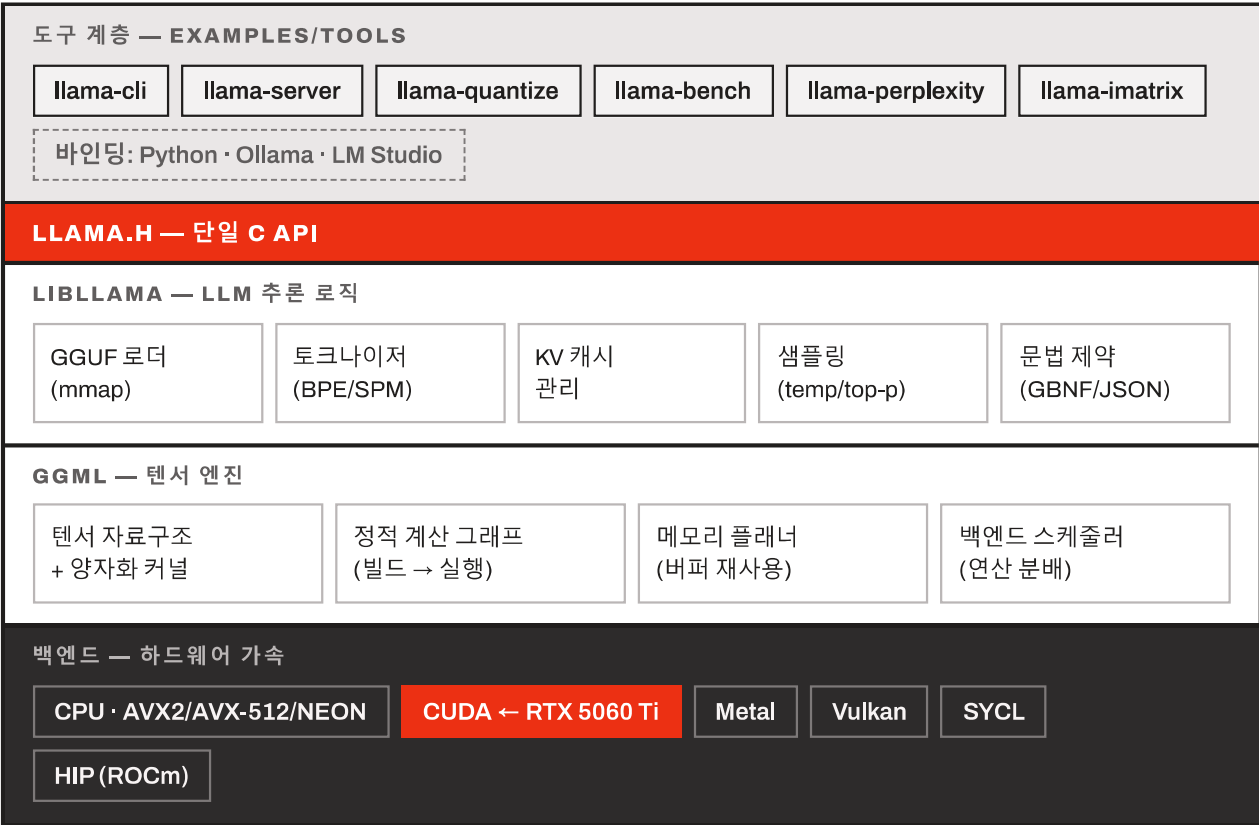


그림 1 — llama.cpp 4계층 구조. 모든 상위 도구는 llama.h C API를 거치고, ggml이 연산을 백엔드에 분배한다.

추론 파이프라인: 토큰이 생성되는 과정

LLM 추론은 두 단계로 나뉜다. **프리필(fill)**은 프롬프트 전체를 병렬 처리해 KV 캐시를 채우는 연산 집약적(compute-bound) 단계이고, **디코드(decode)**는 토큰을 하나씩 생성하는 메모리 대역폭 집약적(memory-bound) 단계다. 디코드 속도는 사실상 "가중치 전체를 얼마나 빨리 읽는가"로 결정되므로, **양자화로 모델이 절반으로 줄면 생성 속도는 약 2배가 된다** — 이것이 양자화가 품질 절충 이상의 의미를 갖는 이유다.

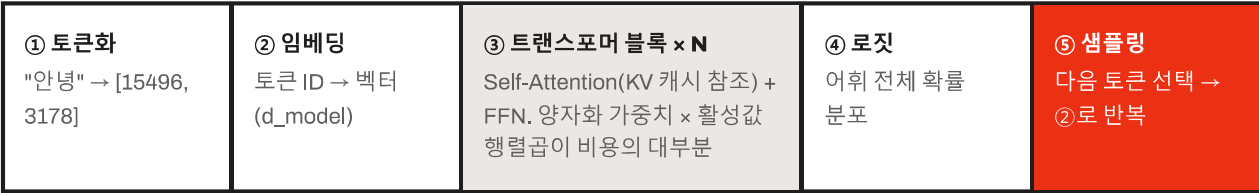


그림 2 — 자기회귀 추론 루프. 생성된 토큰이 다시 입력이 되어 ②~⑤가 토큰마다 반복된다.

03 GGUF 파일 포맷

GGUF(GGML Universal Format)는 2023년 8월 기존 GGML/GGJT 포맷을 대체한 단일 파일 컨테이너다. 핵심 설계 목표는 세 가지: **자기완결성**(토크나이저·하이퍼파라미터·채팅 템플릿까지 파일 하나에 내장 — config.json이 필요 없다), **확장성**(키-값 메타데이터라 새 아키텍처가 나와도 포맷 버전을 올릴 필요가 없다), **mmap 친화성**(텐서 데이터가 32바이트 정렬로 배치되어 파일을 그대로 메모리에 매핑한다).

헤더	매직 넘버 GGUF · 버전(v3) · 텐서 개수 · 메타데이터 KV 개수
메타데이터 (Key-Value)	general.architecture = "qwen3" · llama.block_count = 36 · context_length · embedding_length · RoPE 파라미터 · 토크나이저 어휘 전체 · chat_template · 양자화 버전
텐서 인포 x 텐서 수	이름(blk.0.attn_q.weight) · 차원 · 양자화 타입(Q4_K 등) · 데이터 오프셋
패딩	32바이트 경계 정렬 (general.alignment)
텐서 데이터 (파일의 95%+)	양자화된 가중치 블록이 연속 배치. mmap으로 복사 없이 매핑 → VRAM으로 레이어 단위 업로드

그림 3 — GGUF 파일 레이아웃. 메타데이터만 읽으면 데이터를 로드하지 않고도 모델 정보를 알 수 있다.

파일명에는 관례적으로 양자화 타입이 붙는다: Qwen3-8B-Q4_K_M.gguf . Hugging Face는 GGUF 전용 뷰어를 제공하므로 저장소에서 메타데이터와 텐서 목록을 브라우저로 바로 확인할 수 있다. 하나의 모델 저장소(예: bartowski, unsloth, ggml-org 계정)에 보통 10여 개의 양자화 변형이 함께 올라온다.

04 양자화의 원리

기본 개념: 실수를 정수로 사상하기

양자화(quantization)는 FP16/BF16 실수 가중치를 소수 비트의 정수로 근사하는 것이다. 가장 단순한 형태는 스케일 팩터 하나를 쓰는 선형 양자화다: 가중치 그룹에서 절대값 최댓값을 찾아 $d = \max |w| / 7$ (4비트 부호형이면 $-8 \sim 7$)로 스케일을 정하고, 각 가중치를 $q = \text{round}(w / d)$ 로 저장한다. 복원은 $w \approx q \times d$. 핵심 통찰은 **LLM 가중치가 대체로 0 근처 정규분포를 따르되 드문 이상치(outlier)가 있다**는 점이다. 텐서 전체에 스케일 하나를 쓰면 이상치가 스케일을 키워 나머지 값의 해상도를 망가뜨리므로, llama.cpp는 **블록 양자화** — 가중치를 32개 또는 256개의 작은 블록으로 나눠 블록마다 독립적인 스케일을 두는 방식 — 를 쓴다.

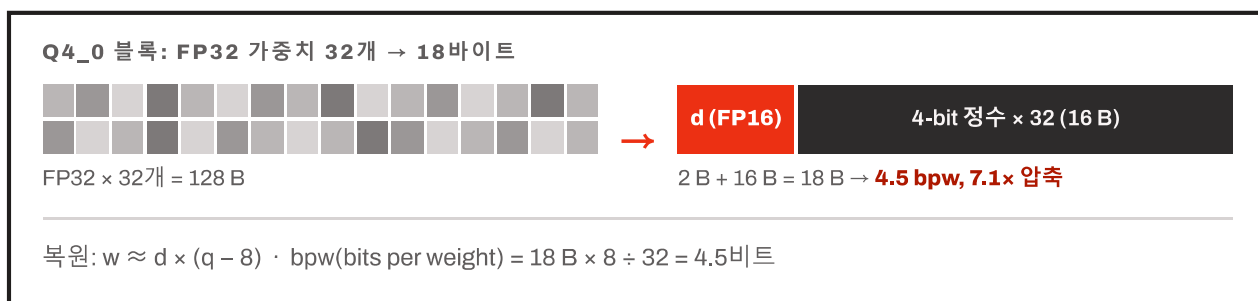


그림 4 — 블록 양자화의 최소 단위. 블록마다 스케일 d 를 따로 두어 이상치의 영향을 32개 이내로 격리한다.

K-quants: 슈퍼블록과 2단계 스케일

2023년 중반 도입된 K-quants(Q2_K~Q6_K)는 블록 양자화를 한 단계 발전시켰다. 256개 가중치를 **슈퍼블록**으로 묶고, 그 안의 서브블록(32개씩 8개)마다 저정밀(6비트) 스케일을 두되 슈퍼블록 전체에 고정밀 FP16 스케일을 공유한다 — 스케일 자체를 양자화하는 **2단계 구조**다. 또한 min 값을 함께 저장하는 비대칭 양자화로 0이 아닌 분포 중심도 정확히 표현한다. 결과적으로 같은 bpw에서 오차가 크게 줄었다. 접미사 **_S/_M/_L**은 믹스 전략을 뜻한다: 예컨대 Q4_K_M은 대부분 Q4_K를 쓰되 민감한 텐서(attention의 V, FFN의 down projection 일부)를 Q6_K로 올린 혼합 구성이다.

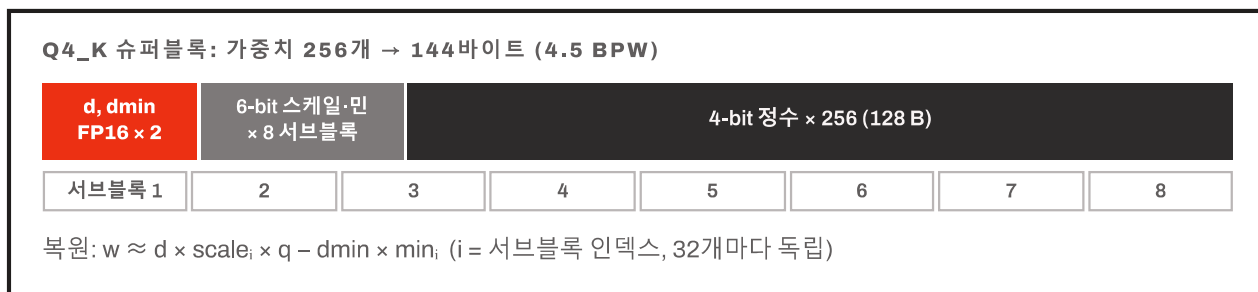


그림 5 — Q4_K 슈퍼블록 구조. 스케일의 스케일(d)을 두는 2단계 계층으로 Q4_0과 같은 4.5 bpw에서 더 낮은 오차를 얻는다.

I-quants와 중요도 행렬(imatrix)

3비트 이하로 내려가면 반올림 기반 방식은 급격히 무너진다. **IQ 계열**(IQ2_XXS~IQ4_XS)은 QuIP# 연구에서 착안해 가중치 8개 묶음을 미리 정의된 **코드북**(E8 격자 기반)의 최근접 벡터로 대체한다 — 개별 반올림이 아닌 벡터 양자화다. 여기에 **중요도 행렬(imatrix)**을 결합한다: 보정 텍스트를 모델에 통과시켜 각 가중치가 실제 활성값과 곱해지는 빈도·크기를 측정하고, 양자화 오차 최소화 시 중요한 가중치에 더 큰 가중치를 두는 것이다. 이는 GPTQ/AWQ와 같은 계열의 아이디어로, 2.5 bpw에서도 실용적인 품질을 가능하게 한다. imatrix는 IQ 타입에는 사실상 필수이고 Q4_K 이상에도 소폭 이득이 있다.

양자화 타입 선택 가이드

타입	BPW	8B 모델 크기	품질 손실	권장 용도
F16 / BF16	16.0	≈ 16.1 GB	기준 (없음)	양자화 원본·품질 기준선
Q8_0	8.5	≈ 8.5 GB	사실상 무손실	VRAM 여유 시 최고 품질
Q6_K	6.56	≈ 6.6 GB	매우 미미	고품질과 크기의 절충
Q5_K_M	5.69	≈ 5.7 GB	미미	품질 우선 일반 사용
Q4_K_M	4.85	≈ 4.9 GB	작음	사실상 표준 — 기본 선택
IQ4_XS	4.25	≈ 4.3 GB	작음 (imatrix 시)	Q4보다 조금 더 작게
Q3_K_M	3.91	≈ 4.0 GB	체감 시작	VRAM이 빠듯할 때
IQ3_XXS	3.06	≈ 3.2 GB	뚜렷하나 사용 가능	큰 모델을 억지로 올릴 때
IQ2_XXS / Q2_K	2.06 / 2.63	≈ 2.2~2.8 GB	큼	70B급을 한 장에 올리는 최후 수단

경험칙: 같은 VRAM이라면 "작은 모델의 고정밀"보다 "큰 모델의 4비트"가 거의 항상 낫다. 8B Q8_0(8.5GB)보다 14B Q4_K_M(9GB)이 대부분의 과제에서 우수하다. 단 3비트 아래로는 모델 크기 이득이 품질 붕괴를 못 이기는 지점이 온다.

05 실습 1 — 환경 구축과 CUDA 빌드

RTX 5060 Ti 하드웨어 이해

RTX 5060 Ti 16GB는 Blackwell 아키텍처(compute capability **sm_120**)로, GDDR7 128-bit에서 약 **448 GB/s**의 메모리 대역폭을 제공한다. 디코드 속도의 이론 상한은 대역폭 ÷ 모델 크기로 근사할 수 있다: 8B Q4_K_M(4.9GB) 기준 약 90 tok/s 상한, 실측은 통상 55~75 tok/s. **반드시 16GB 모델인지 확인하라**(8GB 변형 존재). Blackwell 지원을 위해 **드라이버 570 이상 + CUDA Toolkit 12.8 이상**이 필요하다 — 이전 CUDA로 빌드하면 sm_120 커널이 없어 GPU가 인식되지 않는다.

사전 점검

- ① `nvidia-smi` — 드라이버 570+, "RTX 5060 Ti", 16384MiB 확인
- ② `nvcc --version` — 12.8+ 확인
- ③ Windows라면 WSL2(Ubuntu 24.04) 권장 — 이 문서의 명령은 Linux/WSL2 기준이며 네이티브 Windows는 사전 빌드 바이너리(`llama-*-bin-win-cuda-x64.zip`)를 받아도 된다.

소스 빌드

```
# 빌드 도구 설치 (Ubuntu/WSL2)
sudo apt update && sudo apt install -y build-essential cmake git python3-pip

# 소스 클론
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp

# CUDA 백엔드로 구성 - 12.8+가 sm_120(Blackwell)을 자동 포함
cmake -B build -DGGML_CUDA=ON -DCMAKE_BUILD_TYPE=Release

# 병렬 빌드 (10~20분)
cmake --build build --config Release -j $(nproc)

# 결과물 확인 - build/bin/ 아래에 도구가 생성된다
ls build/bin/ | grep llama
llama-cli llama-server llama-quantize llama-bench llama-perplexity llama-
imatrix ...
```

빌드 검증: `./build/bin/llama-cli --version` 출력에 `ggml_cuda_init: found 1 CUDA devices: Device 0: NVIDIA GeForce RTX 5060 Ti` 가 보여야 한다. 이후 모든 실습에서 `export PATH=$PWD/build/bin:$PATH` 를 해두면 편하다.

참고 — 빌드 옵션 `-DGGML_CUDA_F16=ON` 은 일부 커널을 FP16 연산으로 바꿔 Blackwell에서 소폭 빨라질 수 있다. 여러 GPU 세대용 배포 바이너리가 필요하다면 -
`DCMAKE_CUDA_ARCHITECTURES="86;89;120"` 처럼 명시한다.

06 실습 2 — 모델 다운로드와 첫 추론

llama.cpp는 `-hf` 저장소: 양자화태그 플래그로 Hugging Face에서 GGUF를 직접 받아 캐시(`~/.cache/llama.cpp`)에 저장한다. 첫 모델로는 16GB VRAM에 넉넉히 들어가는 **Qwen3-8B Q4_K_M**($\approx 5\text{GB}$)을 쓴다.

```
# 대화형 실행 - 다운로드가 자동으로 진행된다
llama-cli -hf ggml-org/Qwen3-8B-GGUF:Q4_K_M \
  -ngl 99 \           # 모든 레이어를 GPU에 (99 = 전부)
  -c 8192 \           # 컨텍스트 길이 (KV 캐시 크기 결정)
  -fa on              # FlashAttention - 속도·VRAM 모두 이득 # 단발 프롬프트 실행
llama-cli -hf ggml-org/Qwen3-8B-GGUF:Q4_K_M -ngl 99 \
  -p "양자화가 LLM 추론 속도를 높이는 이유를 세 문장으로 설명해줘." -n 256
```

종료 시 출력되는 타이밍을 읽는 법: `prompt eval time ... tokens per second` 는 프리필 속도(수백~수천 tok/s), `eval time ... tokens per second` 가 체감 생성 속도다. 실행 중 다른 터미널에서 `watch -n1 nvidia-smi` 로 VRAM 점유를 관찰해보라.

주요 파라미터

플래그	의미	권장값 (5060 TI 16GB)
<code>-ngl N</code>	GPU에 올릴 레이어 수	99 (모델이 들어가는 한 전부)
<code>-c N</code>	컨텍스트 길이	8192~16384 (KV 캐시와 트레이드오프)
<code>-fa on</code>	FlashAttention 커널	항상 켜기
<code>-b / -ub</code>	논리/물리 배치 크기	기본값(2048/512)으로 시작
<code>--temp, --top-p</code>	샘플링 다양성	모델 카드 권장값 (Qwen3: 0.6 / 0.95)
<code>-t N</code>	CPU 스레드 수	물리 코어 수 (CPU 오프로딩 시에만 중요)

해볼 것: 양자화 수준 체감 비교

같은 모델의 `:Q8_0` 과 `:Q4_K_M` 태그를 각각 받아 동일 프롬프트(수학 추론, 한국어 작문, 코드 생성)를 넣어보라. 품질 차이는 거의 느껴지지 않지만 생성 속도와 VRAM 점유(`nvidia-smi`)는 뚜렷이 다르다 — 4장의 이론이 실감되는 순간이다.

07 실습 3 — HF 모델을 직접 GGUF로 변환·양자화

이번에는 남이 만든 GGUF를 받는 대신, Hugging Face의 safetensors 원본을 직접 변환한다. 한국어 성능이 좋은 **LGAI-EXAONE/EXAONE-3.5-7.8B-Instruct**를 예제로 쓴다(공개 모델, 로그인 불필요 — Llama 계열은 gated라 `huggingface-cli login` 필요). 전체 흐름:

① HF 원본 다운로드 safetensors + config.json + tokenizer (≈ 16 GB)	② <code>convert_hf_to_gguf.py</code> 텐서 이름 매핑 + 메타데이터 내장 → F16 GGUF	③ llama-imatrix (선택) 보정 텍스트로 중요도 측정	④ llama-quantize F16 → Q4_K_M 등 (몇 분)
---	---	--	--

그림 6 — 변환·양자화 파이프라인. 양자화는 항상 F16/BF16 GGUF에서 출발한다(양자화본 재양자화는 품질 열화).

```
# ① 변환 스크립트 의존성 + 원본 다운로드 (디스크 ≈ 35 GB 확보)
pip install -r requirements.txt huggingface_hub
huggingface-cli download LGAI-EXAONE/EXAONE-3.5-7.8B-Instruct \
  --local-dir ./exaone-3.5-7.8b

# ② safetensors → F16 GGUF (≈ 15.6 GB)
python convert_hf_to_gguf.py ./exaone-3.5-7.8b \
  --outfile exaone-3.5-7.8b-f16.gguf --outtype f16

# ③ 중요도 행렬 — 보정 텍스트는 한국어+영어+코드 혼합 100KB+ 권장
llama-imatrix -m exaone-3.5-7.8b-f16.gguf \
  -f calibration.txt -o exaone-imatrix.dat -ngl 20 -c 512
# F16 원본(15.6GB)은 16GB에 다 안 들어가므로 -ngl로 일부만 GPU에

# ④ 양자화 — imatrix 적용 Q4_K_M과 IQ3_XXS 두 가지 생성
llama-quantize --imatrix exaone-imatrix.dat \
  exaone-3.5-7.8b-f16.gguf exaone-3.5-7.8b-Q4_K_M.gguf Q4_K_M
llama-quantize --imatrix exaone-imatrix.dat \
  exaone-3.5-7.8b-f16.gguf exaone-3.5-7.8b-IQ3_XXS.gguf IQ3_XXS

# 결과 확인 및 실행
ls -lh exaone-*.gguf
15.6G f16 · 4.7G Q4_K_M · 3.1G IQ3_XXS
llama-cli -m exaone-3.5-7.8b-Q4_K_M.gguf -ngl 99 -fa on \
  -p "판교에서 점심 먹기 좋은 식당 스타일을 세 가지 추천해줘."
```

양자화 로그를 관찰하라: 텐서별로 어떤 타입이 지정되는지 출력된다. Q4_K_M에서도 `output.weight` 와 임베딩은 Q6_K로 남는 것을 볼 수 있다 — _M 믹스 전략이 실제로 동작하는 모습이다.

보정 텍스트 팁 — imatrix 품질은 보정 데이터가 실제 사용 분포를 얼마나 닮았는지에 달렸다. 한국어 서비스용이라면 반드시 한국어 텍스트를 절반 이상 포함하라. 커뮤니티 표준으로는 bartowski의 calibration_datav3 텍스트가 널리 쓰인다.

08 실습 4 — llama-server와 OpenAI 호환 API

llama-server는 모델을 상주시키고 HTTP로 서빙한다. OpenAI Chat Completions API와 호환되므로 기존 openai SDK 코드의 `base_url` 만 바꾸면 그대로 동작하고, `http://localhost:8080` 접속 시 내장 채팅 웹 UI도 제공된다.

```
# 서버 기동 - 병렬 4슬롯, 총 컨텍스트 16384 (슬롯당 4096)
llama-server -m exaone-3.5-7.8b-Q4_K_M.gguf \
  -ngl 99 -fa on -c 16384 -np 4 --port 8080

# curl로 호출
curl http://localhost:8080/v1/chat/completions -H "Content-Type: application/json" -d '{
  "messages": [
    {"role": "system", "content": "당신은 간결하게 답하는 조수다."},
    {"role": "user", "content": "GGUF와 safetensors의 차이는?"}
  ],
  "temperature": 0.7, "max_tokens": 256
}'
```

```
# Python - openai SDK를 그대로 사용
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8080/v1", api_key="sk-no-key")

resp = client.chat.completions.create(
    model="local", # 서버가 무시 - 로드된 모델 사용
    messages=[{"role": "user", "content": "블록 양자화를 한 문장으로."}],
    stream=True,
)
for chunk in resp:
    print(chunk.choices[0].delta.content or "", end="")
```

알아둘 서버 기능

연속 배치 + 프롬프트 캐시 -np N으로 동시 요청을 한 배치에 처리. 공통 접두사 (시스템 프롬프트)의 KV는 재사용된다.	구조화 출력 요청에 <code>json_schema</code> 또는 <code>GBNF grammar</code> 를 넣으면 샘플링 단계에서 문법에 맞는 토큰만 허용 — 100% 유효한 JSON을 보장.
스펙큘러티브 디코딩	임베딩·리랭킹 엔드포인트

-md로 작은 드래프트 모델(예: 같은 계열 0.6B)을 붙이면 큰 모델이 초안을 검증만 해 1.5~2배 가속.

--embedding 플래그와 임베딩 모델 GGUF로 /v1/embeddings 제공 — 로컬 RAG 구성 가능.

09 실습 5 — 성능 측정과 VRAM 튜닝

llama-bench: 처리량 측정

```
# pp512 = 프리필 512토큰, tg128 = 생성 128토큰 처리량
llama-bench -m exaone-3.5-7.8b-Q4_K_M.gguf -ngl 99 -fa 1

# 오프로딩 레이어 수에 따른 변화를 한 번에 스캔
llama-bench -m exaone-3.5-7.8b-Q4_K_M.gguf -ngl 0,8,16,24,99 -fa 1
```

llama-perplexity: 양자화 품질 정량화

퍼플렉시티(PPL)는 모델이 검증 텍스트를 얼마나 잘 예측하는지의 지표로, 낮을수록 좋다. 양자화본의 PPL을 F16 기준과 비교하면 품질 손실이 숫자로 보인다. 통상 Q8_0은 +0.1% 이내, Q4_K_M은 +1~3%, IQ3_XXS는 +5~10% 수준.

```
wget https://huggingface.co/datasets/ggml-org/ci/resolve/main/wikitext-2-raw-v1.zip
unzip wikitext-2-raw-v1.zip
llama-perplexity -m exaone-3.5-7.8b-Q4_K_M.gguf \
  -f wikitext-2-raw/wiki.test.raw -ngl 99 -fa on
Final estimate: PPL = 8.0921 +/- 0.05 ← 실습 3에서 만든 각 양자화본을 비교해보라
```

VRAM 예산 세우기 — 16GB의 해부

VRAM은 가중치 + KV 캐시 + 컴퓨트 버퍼 + CUDA 오버헤드로 소비된다. KV 캐시는 컨텍스트 길이에 비례한다(FP16 기준 8B급 모델 약 0.12 GB/1K 토큰 — GQA 구조에 따라 다름). 14B Q4_K_M을 올린 전형적 배분:

가중치 9.0 GB	KV 캐시 2.7 GB	버퍼 1.2 GB	CUDA 1.0 GB	여유 2.1 GB
Qwen3-14B Q4_K_M · 컨텍스트 16K · FP16 KV 기준. 디스플레이 출력용 GPU라면 OS가 0.5~1 GB를 추가로 점유한다.				

그림 7 — RTX 5060 Ti 16GB의 VRAM 배분 예. 컨텍스트를 32K로 늘리면 KV 캐시만 5GB를 넘는다.

VRAM이 부족할 때의 3단계 대응

① **KV 캐시 양자화**: `-ctk q8_0 -ctv q8_0` (FlashAttention 필요)로 KV 캐시를 절반으로 — 품질 손실은 거의 없다. ② **컨텍스트 축소**: 필요 이상의 `-c`는 낭비다. ③ **레이어 오프로딩**: 그래도 부족하면 `-ngl`을 줄여 일부 레이어를 CPU RAM에 둔다. 단 CPU로 내려간 레이어는 수십 배 느리므로 성능은 CPU 쪽 병목에 지배된다 — "가급적 전부 GPU에 들어가는 모델을 고르는 것"이 항상 우선이다.

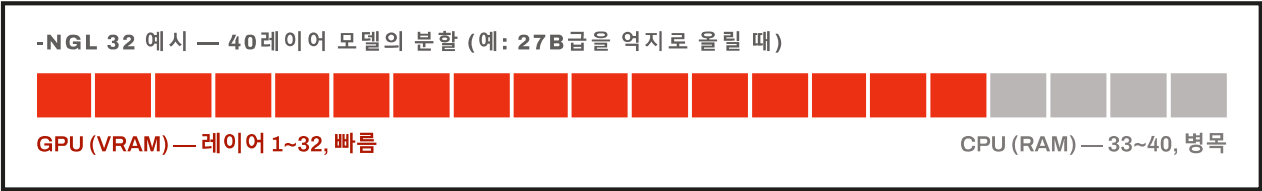


그림 8 — 하이브리드 실행. 전체 속도는 대략 조화평균으로 떨어지므로 CPU 몫이 20%만 되어도 체감 속도는 절반 이하가 된다.

10 부록 — 추천 모델 · 명령어 · 트러블슈팅

RTX 5060 Ti 16GB 추천 모델 (Hugging Face 공개 GGUF)

모델	권장 양자화	VRAM (8K CTX)	특징
Qwen3-14B	Q4_K_M (≈ 9.0 GB)	≈ 11 GB	16GB에서의 종합 성능 스위트스팟, 추론 (thinking) 모드
Qwen3-8B	Q6_K (≈ 6.7 GB)	≈ 9 GB	속도·품질 균형, 긴 컨텍스트 여유
EXAONE 3.5 7.8B	Q4_K_M / Q6_K	≈ 6~9 GB	한국어 최상급 (LG AI연구원)
Gemma 3 12B	Q4_K_M (≈ 7.3 GB)	≈ 10 GB	다국어·이미지 입력(비전) 지원
Llama 3.1 8B	Q5_K_M (≈ 5.7 GB)	≈ 8 GB	생태계·파인튜닝 자료 최다 (gated — HF 승인 필요)
Qwen3-30B-A3B (MoE)	IQ4_XS (≈ 15.3 GB)	부분 오프로딩	활성 3B라 오프로딩해도 빠름 — MoE 실습용
Qwen3-0.6B	Q8_0 (≈ 0.6 GB)	≈ 1.5 GB	스펙큘러티브 디코딩 드래프트용

명령어 치트시트

작업	명령
HF에서 받아 바로 대화	<code>llama-cli -hf 계정/모델-GGUF:Q4_K_M -ngl 99 -fa on</code>
HF 원본 → F16 GGUF	<code>python convert_hf_to_gguf.py ./모델폴더 --outtype f16 --outfile out.gguf</code>
양자화	<code>llama-quantize [--imatrix i.dat] in-f16.gguf out.gguf Q4_K_M</code>
중요도 행렬 생성	<code>llama-imatrix -m f16.gguf -f calib.txt -o i.dat -ngl 20</code>
API 서버	<code>llama-server -m m.gguf -ngl 99 -fa on -c 16384 -np 4</code>
벤치마크	<code>llama-bench -m m.gguf -ngl 99 -fa 1</code>
퍼플렉시티	<code>llama-perplexity -m m.gguf -f wiki.test.raw -ngl 99</code>
GGUF 메타데이터 확인	<code>llama-cli -m m.gguf --no-warmup -p "" -n 0 -v 2>&1 head -50</code>
KV 캐시 절약	<code>-ctk q8_0 -ctv q8_0 (FlashAttention 필요)</code>

트러블슈팅

증상	원인과 해결
GPU를 못 찾음 / found 0 devices	CUDA 12.8 미만으로 빌드됨(sm_120 없음) 또는 드라이버 570 미만 — 툴킷·드라이버 갱신 후 build 폴더 삭제하고 재빌드
cudaMalloc failed / OOM	VRAM 초과 — 컨텍스트 축소 → KV 양자화 → -ngl 축소 순으로 대응 (9장)
생성이 비정상적으로 느림	-ngl 누락(전부 CPU 실행 중)이 가장 흔함. nvidia-smi로 GPU 사용률 확인
출력이 깨짐·무한 반복	채팅 템플릿 불일치 — 최신 GGUF로 교체하거나 --chat-template 지정, 샘플링을 모델 카드 권장값으로
convert 스크립트가 아키텍처 미지원	llama.cpp를 git pull로 최신화 — 신규 모델 지원은 수일 내 머지되는 경우가 대부분
WSL2에서 VRAM이 적게 보임	Windows 쪽 앱이 점유 중 — 디스플레이를 내장 GPU로 돌리거나 불필요한 앱 종료

더 공부할 거리

llama.cpp GitHub(ggml-org/llama.cpp)의 docs 폴더와 Discussions · GGUF 명세(ggml-org/ggml/docs/gguf.md) · K-quants PR #1684, I-quants PR #4773의 설계 토론 · Hugging Face GGUF 문서 및 양자화 계정(bartowski, unsloth, ggml-org) · 논문: GPTQ(2022), AWQ(2023), QuIP#(2024).

핵심 요약

디코드는 메모리 대역폭 게임이다. 양자화는 품질을 조금 내주고 크기·속도·접근성을 사는 거래이며, 16GB GPU에서 그 최적 교환비는 대체로 Q4_K_M 부근에 있다.